

CampusEats — Design Benchmark

CS 543 Web Services · Assignment 2 of 14 — Services, Contracts & Schema

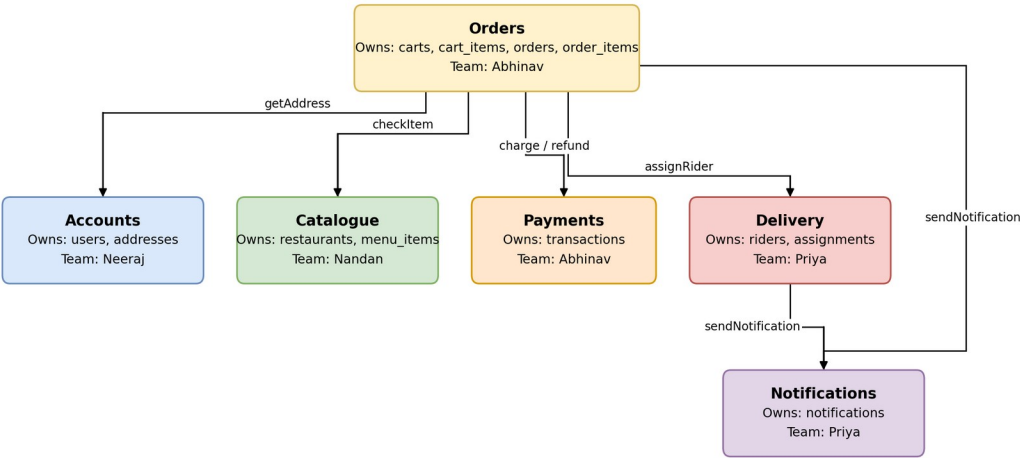
Repository: github.com/ABHINAVX03/CampusEats

Team: Abhinav · Neeraj · Nandan · Priya

Services & Ownership

Six services, four team members. Orders and Payments are paired (placeOrder's core money path); Delivery and Notifications are paired (every delivery-status change fires a notification).

Service	Data Owned	Team Member
Accounts	users, addresses	Neeraj
Catalogue	restaurants, menu_items	Nandan
Orders	carts, cart_items, orders, order_items	Abhinav
Payments	transactions	Abhinav
Delivery	riders, assignments	Priya
Notifications	notifications	Priya



Every cross-service call in the diagram is one operation from the contracts below — nothing calls into a service outside its published contract.

Task 1 — Capabilities

1. Manage student accounts, authentication & delivery addresses
2. Manage restaurant listings & menus (restaurant/admin side)
3. Browse restaurants and menus (student-facing read side)
4. Manage cart and place orders
5. Process payments and refunds
6. Assign a rider and track delivery
7. Send notifications at each order and delivery milestone

Capability 2 is new versus the first draft: Catalogue had read-only operations (`listRestaurants`, `getMenu`, `checkItem`) with no way to ever add a restaurant or menu item — the catalogue would stay permanently empty. Folded into the existing Catalogue service rather than adding a 7th service, to stay within the 3–6 service limit.

Task 3 — Service Contracts

Each table lists the operations other services (or the client) may call. Internal table names, ID formats, and SQL are never exposed — only the operation, its inputs/outputs, and its errors.

1. Accounts

Operation	Input	Output	Errors
registerStudent	name, email, password, phone	userId, status	email already exists
login	email, password	userId, sessionToken	invalid credentials
getProfile	userId	name, email, phone, addresses	user not found
addAddress	userId, street, city, zip, label	addressId	invalid address
updateAddress	userId, addressId, fields	updated addressId	address not found, invalid address
getAddress	userId, addressId	address details	address not found, not owned by user

Bold rows are new vs. the first draft: login was missing entirely; addAddress/getAddress separate address creation from lookup so Orders can verify ownership.

2. Catalogue

Operation	Input	Output	Errors
listRestaurants	area (optional), filter (optional)	list of {id, name, location}	—
getMenu	restaurantId	menu items: {itemId, name, price, available}	restaurant not found
checkItem	itemId	available (bool), price	item not found
addRestaurant	name, location, phone, openingHours	restaurantId	—
addMenuItem	restaurantId, name, price, description, category	itemId	restaurant not found
updateMenuItem	itemId, fields	updated itemId	item not found

Bold rows are new: without them nothing could ever be added to the catalogue.

3. Orders

Operation	Input	Output	Errors
addToCart	studentId, itemId, quantity	cartItemId, cart total	item unavailable
getCart	studentId	cart items, total	—
removeFromCart	studentId, cartItemId	updated cart total	cart item not found

Operation	Input	Output	Errors
placeOrder	studentId, cart, deliveryAddressId, paymentMethodId	orderId, status, total, estimatedMinutes	empty cart, bad address, item unavailable, payment declined, no rider available
getOrder	orderId	order details (status, items, total, delivery info)	order not found
cancelOrder	orderId	new status: cancelled	order already shipped / too late to cancel

Bold rows are new: addToCart existed with no way to view or edit the cart before placing an order.

4. Payments

Operation	Input	Output	Errors
charge	amount, method	transactionId, status	insufficient funds / payment declined
refund	transactionId	status	transaction not found
getTransactionStatus	transactionId	status	transaction not found

getTransactionStatus is new — lets Orders poll a charge instead of only relying on the synchronous return.

5. Delivery

Operation	Input	Output	Errors
assignRider	orderId, deliveryAddress	riderId, estimatedArrival	no rider available
trackOrder	orderId	current location, status	order not assigned
updateDeliveryStatus	assignmentId, status	updated status	assignment not found, invalid transition

updateDeliveryStatus is new — without it an assignment could never move past ASSIGNED; called by the rider-facing client.

6. Notifications

Operation	Input	Output	Errors
sendNotification	userId, orderId, message, type	notificationId, sentAt	user not found
getNotificationHistory	userId	list of notifications	user not found

getNotificationHistory is new — gives students an in-app notification inbox instead of fire-and-forget only.

Task 4 — placeOrder: Full Specification

Purpose

Accepts a student's order request, validates the cart against live catalogue data, charges payment, assigns a rider, and notifies the student — rolling back the charge if a later step fails.

Inputs

studentId, items: [{itemId, qty}] (cart), deliveryAddressId, paymentMethodId. (Authentication itself — resolving the caller's identity/session — is handled upstream of this operation, not passed as an explicit argument.)

Outputs

orderId, status: 'PLACED', total, estimatedMinutes.

Errors

empty cart · item unavailable · invalid or unauthorized delivery address · payment declined · no rider available (new — the original spec called assignRider internally but never listed its failure as a possible placeOrder error).

Internal Steps (hidden from callers)

1. Verify deliveryAddressId belongs to studentId — Accounts.getAddress. (new: previously trusted the client-sent address id with no ownership check)
2. Validate the cart: call Catalogue.checkItem for every line item to confirm availability and fetch the current price.
3. Calculate the total server-side from the fetched Catalogue prices — the client-submitted price, if any, is never trusted.
4. Charge payment — Payments.charge.
5. Create the order record with status 'PLACED', but only after the charge succeeds.
6. Assign a rider — Delivery.assignRider.
7. If assignRider fails (no rider available): call Payments.refund on the transaction just charged, mark the order 'FAILED', and return that error to the caller. (new: the original spec had no compensating action for a failure after payment already succeeded — money would have been taken with no order to show for it.)
8. Send a confirmation notification — Notifications.sendNotification.

Hidden Details

The orders/order_items table structure, ID generation, which specific rider is selected and how, the payment gateway integration used, and the refund-on-failure compensation logic in step 7 — callers only ever see a clean success or a named error.

Task 6 — Validation Against the Five Properties

Where the first draft's design silently assumed something that wasn't actually true, that's called out here as a fail-then-fix rather than smoothed over — the fixes are already folded into the contracts and schema above.

Service	Reachable	Self-contained	Has Contract	Independent	Loosely Coupled	Fix (if failed)
Accounts	✓	✓ owns users/addresses	✓ 6 ops	✓	✓ others call by ID only	—
Catalogue	✓	✓ owns menu data	✓ 6 ops	✓	✓ checkItem is the only lookup path	—
Orders	✓	✓ owns carts/orders	✓ 6 ops	Originally failed: placeOrder trusted a client-supplied address id with no ownership check.	✓ single orchestrator; no other service calls back into Orders	Fixed — added Accounts.getAddress verification as step 1 of placeOrder.
Payments	✓	Originally partial: refund existed but nothing linked a refund back to the charge it reversed.	✓ 3 ops	✓ can swap gateway internally	✓ others only call charge/refund	Fixed — added refund_of_transaction_id self-reference in the transactions table.
Delivery	✓	Originally failed: no operation could move an assignment past status ASSIGNED.	✓ 3 ops	✓ can change assignment logic freely	✓ others call assignRider/trackOrder	Fixed — added updateDeliveryStatus for the rider-facing client to call.
Notifications	Originally partial: only Orders called it, so delivery-stage events (rider assigned, arriving, delivered) had no way to reach the student.	✓ owns notification log	✓ 2 ops	✓ can change delivery channel freely	Fixed — added a Delivery → Notifications call for delivery-stage events.	Fixed — see Reachable column.